

# Royal Siege

by

Paul Hodges

March 31, 2025

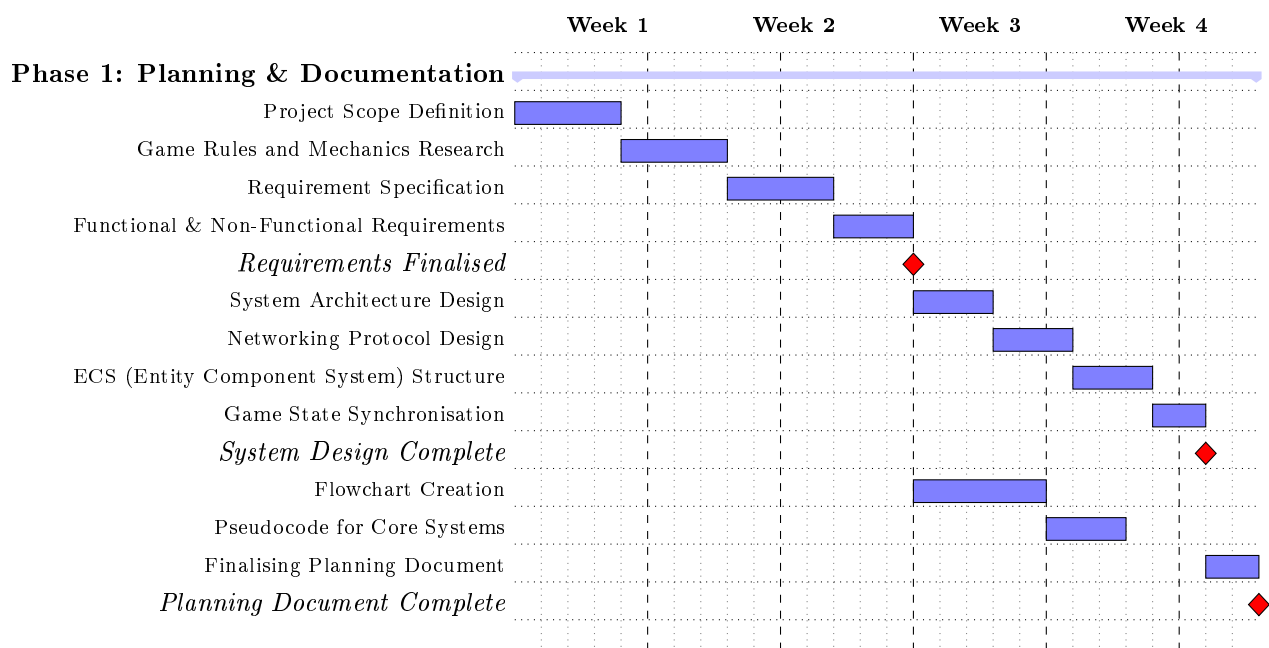
# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Development Schedule</b>                      | <b>3</b>  |
| 1.1      | Planning and Documentation . . . . .             | 3         |
| 1.2      | Programming and Development . . . . .            | 3         |
| <b>2</b> | <b>Problem Outline</b>                           | <b>3</b>  |
| <b>3</b> | <b>Problem Description</b>                       | <b>4</b>  |
| 3.1      | Game Overview . . . . .                          | 4         |
| 3.2      | Game Objectives . . . . .                        | 4         |
| 3.3      | Core Game Rules . . . . .                        | 4         |
| 3.4      | Gameplay Flow . . . . .                          | 5         |
| 3.5      | Scoring System . . . . .                         | 6         |
| 3.6      | Winner Determination . . . . .                   | 6         |
| 3.7      | Advanced Game Mechanics and Strategies . . . . . | 6         |
| <b>4</b> | <b>Requirements</b>                              | <b>7</b>  |
| 4.1      | Functional Requirements . . . . .                | 7         |
| 4.2      | Non-Functional Requirements . . . . .            | 10        |
| <b>5</b> | <b>Design</b>                                    | <b>11</b> |
| 5.1      | Screen Layouts . . . . .                         | 11        |
| 5.1.1    | Main Menu . . . . .                              | 11        |
| 5.1.2    | Matchmaking screen . . . . .                     | 11        |
| 5.1.3    | Battle Arena . . . . .                           | 12        |
| 5.1.4    | Deck Builder . . . . .                           | 12        |
| 5.1.5    | Victory . . . . .                                | 13        |
| 5.2      | Core System Design . . . . .                     | 13        |
| 5.2.1    | Engine . . . . .                                 | 13        |
| 5.2.1.1  | ECS Architecture . . . . .                       | 13        |
| 5.2.1.2  | 2D/3D Cameras . . . . .                          | 13        |
| 5.2.1.3  | GameObjects and Components . . . . .             | 13        |
| 5.2.1.4  | Layered Sprite Rendering . . . . .               | 14        |
| 5.2.1.5  | Rendering . . . . .                              | 14        |
| 5.2.1.6  | Shared State . . . . .                           | 14        |
| 5.2.2    | Network . . . . .                                | 15        |
| 5.2.2.1  | Custom Binary Protocol . . . . .                 | 15        |
| 5.2.2.2  | Client-Server Architecture . . . . .             | 15        |
| 5.2.2.3  | Asynchronous Sockets . . . . .                   | 15        |
| 5.2.2.4  | Packet Listener and Broadcaster . . . . .        | 15        |
| 5.2.3    | 2D with 3D Illusion . . . . .                    | 15        |
| 5.2.3.1  | Layered Sprites . . . . .                        | 15        |
| 5.2.3.2  | Shadow Rendering . . . . .                       | 15        |
| 5.2.3.3  | Perspective Scaling . . . . .                    | 15        |
| <b>6</b> | <b>Pseudocode Implementation</b>                 | <b>16</b> |
| 6.1      | Algorithm Flowcharts . . . . .                   | 31        |
| 6.1.1    | Main Server Loop Flowchart . . . . .             | 31        |
| 6.1.2    | Card Deployment Process Flowchart . . . . .      | 32        |
| 6.1.3    | Battle Update Logic Flowchart . . . . .          | 33        |
| 6.1.4    | Target Selection Algorithm Flowchart . . . . .   | 34        |
| 6.1.5    | Chest Reward System Flowchart . . . . .          | 35        |

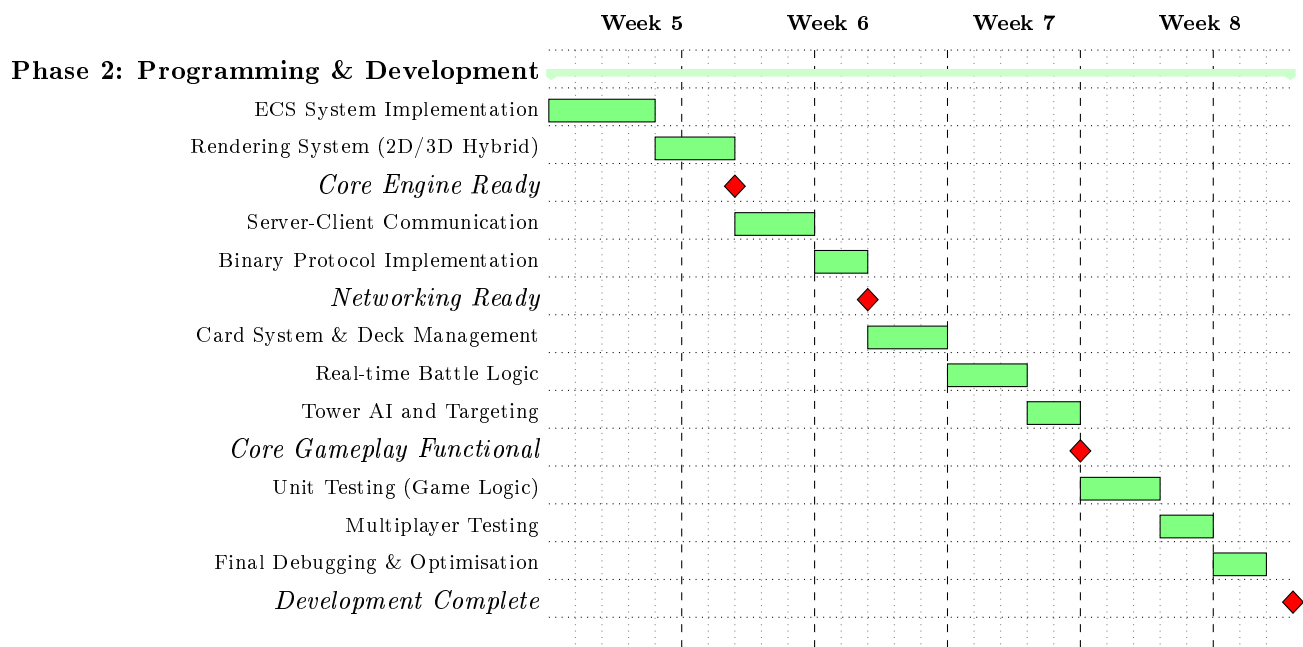
|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>7</b> | <b>Trace Tables</b>                               | <b>36</b> |
| 7.1      | Main Server Loop . . . . .                        | 36        |
| 7.2      | Player Handling (HANDLE_PLAYER) . . . . .         | 36        |
| 7.3      | Request Routing (HANDLE_PLAYER_REQUEST) . . . . . | 36        |
| 7.4      | Matchmaking Process . . . . .                     | 36        |
| 7.5      | Game Update Thread (UpdateAllGames) . . . . .     | 36        |
| 7.6      | Game Update Logic (UPDATE_GAME) . . . . .         | 37        |
| 7.7      | Card Deployment Process . . . . .                 | 37        |
| 7.8      | Target Selection Algorithm . . . . .              | 37        |

# 1 Development Schedule

## 1.1 Planning and Documentation



## 1.2 Programming and Development



# 2 Problem Outline

The purpose of this project is to develop a simplified but functional online version of Clash Royale using Python. This real-time strategy game implementation will focus on core gameplay mechanics and network functionality.

The game will allow players to:

- Engage in real-time, online strategic battles against other players
- Build and customise card decks from their collection
- Progress through a skill-based arena system
- Earn rewards through a chest system to expand their card collection
- Experience the core strategic elements that make Clash Royale engaging

## 3 Problem Description

### 3.1 Game Overview

Clash Royale is a fast-paced, real-time strategy (RTS) game that incorporates elements from multiple genres:

- Collectible Card Game (CCG): Players build and customise decks from a pool of collectible cards. Each card corresponds to a troop, spell, or building, each with its own unique attributes and effects.
- Tower Defense: Players must defend their own towers (two Princess Towers and one King Tower) from enemy attacks while simultaneously attempting to destroy opponent towers.
- Multiplayer Online Battle Arena (MOBA): Two players face off in a real-time battle, managing resources (elixir) and positioning units tactically on a symmetrical arena.

Key aspects include:

- Short Match Durations: Standard matches last 3 minutes, potentially extended by sudden-death overtime.
- Focus on Tactics and Timing: Players must carefully manage elixir and deploy units at optimal moments.
- Intense Head-to-Head Combat: Each battle is a direct confrontation with another human opponent.

### 3.2 Game Objectives

- Primary Goal: Destroy more towers than the opponent before time runs out, or immediately win by destroying the enemy King Tower.
- Crown System:
  - Destroying a Princess Tower yields 1 crown.
  - Destroying the King Tower yields 3 crowns, resulting in an instant victory.
- Overtime: If both players have an equal number of crowns at the end of 3 minutes, the match enters a sudden-death overtime (up to 2 minutes). The first player to gain a lead in crowns during overtime wins immediately.
- Draw: If neither player gains an advantage by the end of overtime, the match ends in a draw.

### 3.3 Core Game Rules

#### 1. Deck and Card Mechanics:

- Each player constructs an 8-card deck. At match start, 4 cards are randomly drawn from the deck. When a card is played, it is replaced by another from the remaining deck to maintain a 4-card hand.
- Cards have different rarities (Common, Rare, Epic, Legendary) and can be upgraded to increase stats.

#### 2. Elixir Resource:

- Elixir is the primary resource used to deploy cards.
- Elixir regenerates at a base rate of 1 elixir per 2.8 seconds.
- The maximum elixir capacity is 10 units.
- During the first minute of overtime, elixir generation doubles (about 1 elixir per 1.4 seconds), and it can triple in the second minute (about 1 elixir per 0.9 seconds), depending on the game mode.

### 3. Deployment Rules:

- Troops and buildings must generally be deployed on the player's side of the arena.
- Certain spells can be deployed anywhere (e.g., damage spells) as allowed by individual card mechanics.
- Cards cannot be deployed if the player does not have enough elixir; partial elixir usage is not possible.

### 4. Unit Behavior:

- Troops move automatically toward enemy units or towers following predefined paths, typically targeting the nearest threat or specific priority targets (e.g., building-focused troops).
- Air and ground units have separate movement planes, meaning air troops may bypass ground-only defenses if not countered by anti-air.
- Troops have attributes such as health, damage per second (DPS), movement speed, attack speed, attack range, and special abilities (e.g., area damage).

### 5. Match Duration and Victory:

- The standard match duration is 3 minutes.
- If the crowns are tied after 3 minutes, overtime begins with sudden-death rules.
- Destroying an opponent's King Tower results in an immediate victory regardless of time or remaining towers.

## 3.4 Gameplay Flow

### 1. Matchmaking:

- Players are paired based on trophy count, ensuring comparable skill levels (within acceptable matchmaking ranges).
- Additional factors (e.g., hidden matchmaking rating, player level) can be considered to improve match quality.

### 2. Battle Start:

- Each player begins with around 5 elixir (default) and 4 randomly drawn cards from their 8-card deck.
- A short countdown (e.g., 3 seconds) precedes the start of the match, allowing players to view their starting hand.

### 3. Real-Time Combat:

- Players deploy troops, buildings, or spells using elixir.
- Units spawn at the chosen deployment location and automatically engage enemy targets.
- Spell effects apply immediately if valid (e.g., damage, stun, or buff).

### 4. Tower Destruction:

- When a tower's HP reaches zero, it is destroyed. The opposing player gains crowns accordingly.
- Destruction of the King Tower instantly ends the match with a 3-crown victory for the attacker.

### 5. Overtime and Conclusion:

- If crowns are tied at the end of regulation, the match enters overtime.
- Elixir generation rate may double or triple during overtime for faster-paced final engagements.
- If the tie persists after overtime, the match concludes in a draw.
- Players are shown a results screen detailing crowns, trophies gained/lost, and any rewards.

### 3.5 Scoring System

- Princess Tower Destruction: +1 crown per destroyed Princess Tower.
- King Tower Destruction: +3 crowns; match ends immediately.
- Time Expiration: If no King Tower is destroyed, the player with the most crowns wins.
- Draw: If the match remains tied after overtime, both players neither gain nor lose trophies (in most modes).
- Trophy Adjustment: Winners gain trophies; losers lose trophies. Draws typically result in no or minimal trophy change.

### 3.6 Winner Determination

- Instant Victory: Destroying the opponent's King Tower at any time.
- Crown Comparison: If neither King Tower is destroyed, the number of Princess Towers destroyed determines the winner.
- Overtime Mechanic: Sudden death rules apply, where the next destroyed tower (or difference in crowns) decides the match outcome.
- Draw: If neither player secures a lead by the end of overtime, the match is declared a draw.

### 3.7 Advanced Game Mechanics and Strategies

- Elixir Management: Maintaining a balance between offence and defence, ensuring players do not overspend or waste potential elixir generation.
- Counter Deployment: Identifying key counters for popular units (e.g., swarm troops counter single-target tanks, ranged troops counter slow-moving units, etc.).
- Card Synergies: Combining cards (e.g., tank + high-DPS unit) for effective pushes. Air-ground synergy is particularly important.
- Lane Pressure: Forcing opponents to split their defences by attacking both left and right lanes strategically.
- Elixir Counting: Skilled players track the approximate elixir of the opponent to gauge when to launch a heavy push.
- Card Cycle Management: Players cycle quickly to return to core cards (win conditions or essential counters).
- Tower Trading: Allowing a weaker tower to fall can free elixir for a stronger, faster offensive counter-push.
- Spell Value: Using damaging spells to hit multiple units at once, or a unit plus a tower, maximises elixir efficiency.
- Unit Placement: Subtle differences in troop deployment positions can dramatically change the outcome of an interaction (kiting, path manipulation).
- Timing: Well-timed deployments catch opponents low on elixir or out of defensive options, leading to big elixir swings.

## 4 Requirements

### 4.1 Functional Requirements

#### FR1: Server-Client Architecture

- Real-Time Multiplayer Connection:
  - The system shall use a persistent connection protocol (e.g., WebSockets) to ensure minimal latency and real-time data exchange.
  - Each client connection shall be encrypted (e.g., TLS/SSL) to protect sensitive information.
- Centralised Game Logic:
  - The server shall manage all core gameplay elements (e.g., elixir ticks, card interactions, damage calculations) for fairness and anti-cheat purposes.
  - Clients shall send deployment/spell actions, and the server shall validate and execute these actions within game state updates.
- Scalability and Load Balancing:
  - The architecture must allow horizontal scaling with load balancers to distribute match-making requests and battle servers as the player base grows.
  - A master server may handle matchmaking, while additional battle servers simulate individual matches.
- Connection Handling:
  - The system must manage client disconnections gracefully, allowing rejoin within a defined grace period (e.g., 30 seconds).
  - Idle sessions shall be terminated to free server resources.

#### FR2: Card Management System

- Card Collection:
  - Each player has a personal inventory of collected cards, stored and displayed in a dedicated UI (e.g., “Card Collection” screen).
  - Cards are grouped by rarity (Common, Rare, Epic, Legendary) and may also be sorted by elixir cost, usage frequency, or level.
- Deck Building:
  - Players must be able to create, edit, and save multiple decks of exactly 8 cards.
  - The deck-building interface shall validate that the deck has no illegal combinations if such constraints exist (e.g., no duplicates in standard modes).
  - Players shall have the option to rename their decks for quick identification.
- Card Upgrades:
  - Each card has level-based attributes. Upgrades require gold and a certain number of card copies.
  - Players shall receive a notification or prompt when they have enough copies and gold to perform an upgrade.
  - Upgrading a card increases hitpoints, damage, or other relevant stats depending on the card type.
- Card Unlocking:
  - New cards become available as players reach certain trophy thresholds (arena levels).
  - Players can obtain these cards through chests, shop purchases (if implemented), or special events.

#### FR3: Real-time Battle Simulation

- Game State Updates:
  - The server shall send synchronised updates (positions, health, elixir, etc.) at a rate of at least 10 ticks per second.



- Clients shall interpolate or render the game state for smooth animations on the front end.
- Deployment and Combat Resolution:
  - Each deployment request from the client must include card ID, position, and current elixir count.
  - The server validates and executes the deployment if the player has sufficient elixir and the placement is valid.
  - Combat is automatically resolved by the server, computing damage based on unit stats, interactions, and spells.
- Collisions and Pathfinding:
  - The system shall implement a simplified pathfinding algorithm to prevent units from occupying the same space.
  - If two units meet head-on, one or both may adjust pathing slightly to avoid overlap, or a priority system may determine which unit yields ground.
- Overtime Mechanics:
  - Once regulation time ends in a tie, the system triggers sudden death and adjusts the elixir regeneration rate.
  - If a tower is destroyed during sudden death, the match ends instantly.
  - If the tie persists after the defined overtime period, the match is declared a draw.

**FR4: Progression System**

- Trophy-Based Ranking:
  - Each victory adds trophies to the player's total, and each loss subtracts trophies.
  - The trophy adjustment may consider the trophy difference between opponents, awarding or deducting more/less accordingly.
- Arena Unlocks:
  - Each trophy milestone corresponds to a new arena, with unique visuals and an expanding pool of unlockable cards.
  - The UI shall highlight newly unlocked features or cards when a player enters a new arena.
- Matchmaking and Leagues:
  - At higher trophy counts, players may enter league divisions (e.g., Challenger, Master, Champion), each conferring distinct rewards.
  - Seasonal resets may occur in leagues, reducing trophies to a certain baseline and distributing end-of-season rewards.

**FR5: Reward System**

- Chest Types and Mechanics:
  - Victory chests are acquired after each win, each having a timer to unlock (ranging from a few hours to a full day).
  - Chest rewards scale with arena level, offering more cards and gold at higher tiers.
  - Special chest types (e.g., Giant, Magical, Legendary) have different probability distributions for rare cards.
- Unlocking Process:
  - A player may only have a limited number of chest slots (e.g., 4). Additional wins while slots are full do not grant new chests.
  - Players can spend premium currency (gems) to speed up or instantly finish chest unlocks.
  - Upon unlock, players receive the content immediately, with any new cards indicated visually in the UI.
- Additional Rewards:
  - Daily login bonuses, quests, or event chests may provide extra gold, cards, or currency.
  - The system can incorporate battle pass mechanics with tiered rewards for active players.

**FR6: Clan System**

- **Clan Creation and Membership:**
  - Any player meeting a minimum requirement (e.g., certain trophy threshold) can create a clan.
  - Clans can have a maximum capacity (e.g., 50 members), with join requests requiring leader or co-leader approval.
- **Clan Roles and Permissions:**
  - Leader can promote/demote members, accept/kick members, and modify clan settings.
  - Co-leader has similar privileges but may not be able to transfer leadership.
  - Elder typically can accept or reject join requests and assist in management.
  - Members can donate/request cards and participate in clan events.
- **Clan Chat and Donations:**
  - A real-time chat allows members to communicate, share strategies, and coordinate.
  - Players can request specific card donations, and other clan members can donate from their own card pool, earning gold or experience in return.
- **Clan Wars or Clan Events (Optional Advanced Feature):**
  - Clans can compete in organised clan wars or events for collective rewards (cards, gold, clan trophies).
  - Each participant's contributions in a clan war might be limited (e.g., a certain number of attacks per day).

**FR7: User Authentication**

- **Registration and Secure Login:**
  - Users must register with a unique username/email. A secure password mechanism with complexity rules (minimum length, character diversity) is required.
  - Passwords shall be stored using salted hashing algorithms (e.g., bcrypt, Argon2) to protect credentials in case of database breaches.
- **Session Management:**
  - After login, the server issues a session token (e.g., JWT) which must be presented with each subsequent request.
  - Session tokens expire after a predefined period (e.g., 24 hours or configurable), or upon explicit logout.
  - Failed login attempts shall be rate-limited or locked after repeated failures to mitigate brute-force attacks.

**FR8: Database Integration**

- **Data Storage:**
  - A relational or NoSQL database will store player profiles, trophies, card inventories, deck configurations, clan information, and transaction logs.
  - Battle replays or partial replay data (moves, timestamps) may be stored for a set period to allow replay features.
- **Atomic Transactions:**
  - High-value operations (e.g., purchases, chest openings, card upgrades) shall be transactional to ensure data consistency (ACID properties).
  - Rollback logic should revert partial updates if an operation fails partway.
- **Backup and Recovery:**
  - Automated incremental and full backups of the database must be performed regularly.
  - Disaster recovery procedures shall be documented, ensuring minimal downtime and data loss in case of critical failures.
- **Scalability Considerations:**
  - The database shall support sharding or replication to accommodate a growing user base.
  - Indexing on frequently queried fields (e.g., user IDs, trophy counts) shall be optimised to reduce query latency.

## 4.2 Non-Functional Requirements

### NFR1: Performance

- **Concurrent Users:** The system shall support at least 100 concurrent players seamlessly, with a roadmap to scale to thousands if the user base grows.
- **Response Time:** Average server response time for high-frequency operations (e.g., deploying a card) shall not exceed 200 ms under typical load.
- **Tick Rate:** The battle server shall update game state at least 10 times per second (ideally 20+ for smoother experience).
- **Client Frame Rate:** The client shall maintain a minimum of 30 FPS on recommended device specifications for fluid animations.

### NFR2: Reliability

- **Uptime Guarantee:** The system should achieve 99% or higher uptime annually, excluding scheduled maintenance.
- **Failover and Recovery:** In case of server crashes, automated restart processes must resume the service with minimal downtime and no data corruption.
- **Reconnection Logic:** If the client disconnects due to network issues, it should automatically attempt to reconnect, rejoining an active match if still in progress.
- **Data Integrity Checks:** The database shall regularly run integrity checks (checksums or similar) to detect and fix corruption.

### NFR3: Scalability

- **Horizontal Scaling:** The system architecture must allow adding more servers to handle increased load without significant refactoring.
- **Matchmaking Algorithm:** The matchmaking system must handle larger player populations efficiently, maintaining reasonable queue times and matching fairness.
- **Database Sharding/Replication:** As the player base grows, the database layer should scale by distributing data across multiple nodes or using replication strategies.

### NFR4: Maintainability

- **Coding Standards:** The codebase shall conform to PEP 8 (Python) or a recognised standard (if other languages are used) to ensure readability and uniformity.
- **Modular Architecture:** Features like matchmaking, battle simulation, user authentication, and clan management should be modularised or separated into microservices.
- **Documentation and Comments:**
  - Detailed API documentation (using tools like Swagger/OpenAPI) should be provided for each service endpoint.
  - Inline comments and design notes shall clarify complex logic for future maintainers.
- **Logging and Monitoring:**
  - A standardised logging format with log levels (debug, info, warning, error) must be used across services.
  - Monitoring tools (e.g., Prometheus, Grafana) shall be in place to track CPU usage, memory, network latency, etc.

### NFR5: Security

- **Credential Storage:** All user passwords must be stored using salted hashing (bcrypt, Argon2, PBKDF2). Plaintext passwords are strictly forbidden.
- **Encryption:** All communication between client and server must be secured with TLS/SSL to protect data in transit.
- **Input Validation:** The server must rigorously validate client inputs (e.g., deployment commands, user IDs, trophy updates) to prevent injection attacks or exploits.

- Attack Mitigation:
  - Protect against common web vulnerabilities (SQL injection, cross-site scripting, CSRF) via standard frameworks and best practices.
  - Rate-limit login attempts and critical actions to prevent brute force or DDoS style attacks.

## 5 Design

### 5.1 Screen Layouts

#### 5.1.1 Main Menu



Figure 1: Main Menu Screen

#### 5.1.2 Matchmaking screen

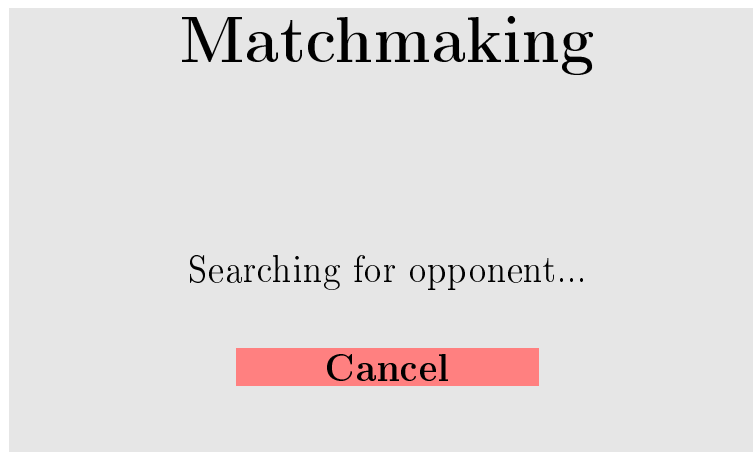


Figure 2: Matchmaking Screen

### 5.1.3 Battle Arena

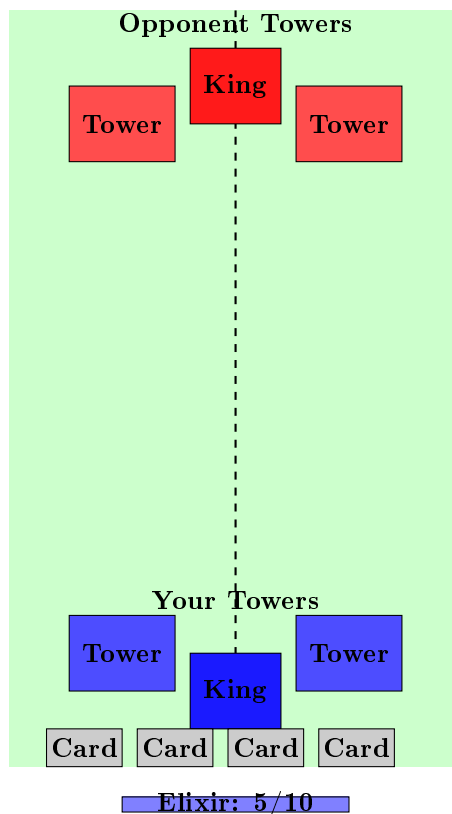


Figure 3: Vertical Battle Arena Layout

### 5.1.4 Deck Builder

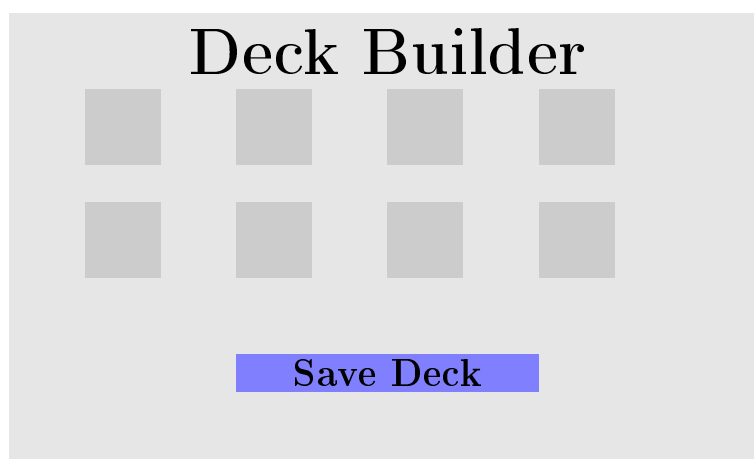


Figure 4: Deck Builder Screen

### 5.1.5 Victory

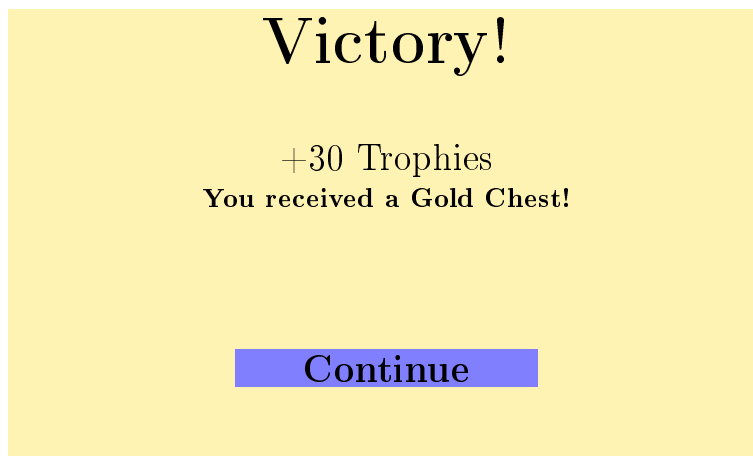


Figure 5: Victory Screen

## 5.2 Core System Design

The game will be built using a custom Entity Component System (ECS) engine and a custom network implementation. The visual style will be 2D with a simulated 3D look achieved through layered sprites and shadow rendering.

### 5.2.1 Engine

The engine is written in Python, based on the design off of one of my previous Rust engines, "Oxidized." Key features include:

#### 5.2.1.1 ECS Architecture

The Entity Component System (ECS) architecture separates logic (components) from entities (system). Entities are simple IDs that contain components, and components execute logic when told to through the pipeline (see 5.2.1.6). Components operate through entities that have the component attached. Components are separate, even from other components of the same type. This design promotes data-oriented programming, improving cache efficiency and making the code more modular and maintainable. In this Python implementation, I use dictionaries and lists to store and manage entities and their related components. This decoupling of different logic systems and containers allows for flexible entity composition and avoids the complexities of traditional inheritance-based object oriented programming.

#### 5.2.1.2 2D/3D Cameras

The engine will support both orthographic and perspective cameras. Orthographic cameras provide a parallel projection, useful for classic 2D views where depth isn't emphasised. Perspective cameras simulate real-world vision, where objects appear smaller as they recede into the distance. This is crucial for creating the simulated 3D look. The camera implementation will likely involve a transformation matrix that converts world coordinates to screen coordinates. For perspective cameras, this matrix will include a perspective projection component, while orthographic cameras will use a simple scaling and translation matrix. I have functions to easily switch between camera types, and to adjust the field of view of the perspective camera.

#### 5.2.1.3 GameObjects and Components

GameObjects serve as containers for components. By assigning different combinations of components to a GameObject, I can create a wide variety of entities with unique behaviors. For example, a "Player" GameObject might have "Position," "Velocity," "Sprite," "Input," and "Health" components. A "Tree" GameObject might have "Position" and "Sprite" components. This modularity allows for easy creation

and modification of game entities without modifying core engine code. The component data will be stored in data structures that are easily accessible by the systems.

#### 5.2.1.4 Layered Sprite Rendering

To create the illusion of depth in a 2D environment, I will use layered sprites. Sprites representing objects closer to the "camera" will be rendered on top of sprites representing objects further away. I will implement a sorting mechanism based on the Y-coordinate (or a custom depth value) of the entities to determine the rendering order. This will allow for overlapping sprites to create a sense of depth. Furthermore, I will have the capability to assign a layer number to each sprite, allowing for more manual control over the draw order, and enabling the creation of backgrounds, midgrounds, and foregrounds.

#### 5.2.1.5 Rendering

The rendering in my Python engine does not use conventional libraries like `pygame`. Instead, I developed a custom C++ library that utilises SDL2 for drawing to the screen. This library also provides advanced functionality for handling textures, images, and keyboard input. To enable usage in Python, I create bindings using `pybind11`. The process involves compiling the library separately for Windows and Linux, then compiling the bindings, and finally linking them together to produce a CPython-compatible library. Additionally, I generate a stub file for Language Server Protocol (LSP) support using `stubgen`, a tool included with Python.

#### 5.2.1.6 Shared State

Throughout the codebase, the way that state is shared between threads and different classes is through a custom made "pipeline". The pipeline follows the same structure as a data bus, in which data can be sent down the pipe by anyone through just having a reference to the class and "consumed" by any listener appended to it. The state is managed by two pipelines, one which communicates state data, and one that functions as an event loop, the event loop is used for many other things but in the case of program state management it is used to communicate a request for a state update and then the other pipeline will contain a message of the new state, with a correlating id. The messages are transmitted as "frames" in which a single frame contains:

```
@total_ordering
@dataclass(order=False)
class Frame(Generic[T]):
    data: T = field(compare=False)

    available_at: float = field(default_factory=time.time)
    priority: int = 0
    id: int = 0

    metadata: Optional[Dict[str, Any]] = field(default=None, compare=False)
    annotations: Dict[str, Any] = field(default_factory=dict, compare=False)
    timestamp: float = field(default_factory=time.time, compare=False)
    sender_id: Optional[str] = field(default=None, compare=False)
    expire_at: Optional[float] = field(default=None, compare=False)
    retry_count: int = field(default=0, compare=False)
    delivered_to: List[str] = field(default_factory=list, compare=False)
    correlation_id: Optional[str] = field(default=None, compare=False)
    topic: Optional[str] = field(default=None, compare=False)
    is_response: bool = field(default=False, compare=False)
```

A frame contains data of type T, which is a generic saying that the frame can only contain data that is a specific type. This is used because one pipeline can only transmit frames of one type to avoid typemismatch and runtime errors. Python doesn't natively support typesafety as it is not compiled and is interpreted on the fly. There is a large amount of typehints within the code so I can use external tools to check for mismatches and prevent a large amount of common runtime errors. There is other data contained in a frame that is managed by the pipeline, like delivered to. Frames check for acknowledgement by other attached listeners to make sure that there is not a mismatch between threads.

### 5.2.2 Network

The network uses custom socket communication with binary data transmission:

#### 5.2.2.1 Custom Binary Protocol

Instead of relying on human-readable text-based protocols, I use a custom binary protocol. This protocol will define the structure of packets sent between the client and server, specifying the types and sizes of data fields. Binary protocols are more compact and efficient, reducing network overhead and latency. I will use python's struct module to pack and unpack data into binary format. I will define packet types, and each packet type will have a defined data structure.

#### 5.2.2.2 Client-Server Architecture

I will implement a client-server architecture, where the server acts as the authoritative source of game state. Clients send input to the server, and the server processes the input, updates the game state, and sends updates back to the clients. This architecture helps prevent cheating and ensures consistency across all clients. The server will maintain the master copy of the game world, and the clients will maintain local copies that are kept in sync with the server.

#### 5.2.2.3 Asynchronous Sockets

To avoid blocking the main game loop, I will use asynchronous sockets. Asynchronous sockets allow the game to continue running while waiting for network data. I will use python's asyncio library or similar, to handle the asynchronous network operations. This will prevent the game from freezing or becoming unresponsive during network communication.

#### 5.2.2.4 Packet Listener and Broadcaster

The server will have a packet listener to receive incoming packets from clients and a packet broadcaster to send packets to all connected clients. The listener will parse incoming packets and dispatch them to the appropriate game logic. The broadcaster will efficiently distribute game state updates to all clients, minimising network traffic. I will implement packet queuing and throttling to prevent network congestion. The broadcaster will only send data that has changed, to reduce the amount of data sent.

### 5.2.3 2D with 3D Illusion

The 3D look is achieved through:

#### 5.2.3.1 Layered Sprites

As mentioned earlier, layered sprites are crucial for creating the illusion of depth. By rendering sprites in a specific order based on their perceived depth, I can simulate the effect of objects being in front of or behind each other. This will involve sorting sprites by their Y-coordinate or a custom depth value before rendering. I will use a depth buffer, or an equivalent sort algorithm, to ensure that sprites are drawn in the correct order.

#### 5.2.3.2 Shadow Rendering

Adding shadows to objects can significantly enhance depth perception. I will implement shadow rendering by projecting a simplified silhouette of each object onto the ground. The darkness and length of the shadow will vary based on the object's height and the light source's position. This will give objects a sense of grounding and add visual depth to the scene. I will use sprite masks to create the shadow silhouettes.

#### 5.2.3.3 Perspective Scaling

To further enhance the 3D illusion, I will implement perspective scaling. Objects farther away from the "camera" will appear smaller, while objects closer will appear larger. This will be achieved by adjusting the sprite's scale based on its distance from the camera. I will use a perspective projection matrix to



calculate the scale factor. This will be done in conjunction with the Layered Sprite rendering, to create a believable 3D effect.

## 6 Pseudocode Implementation

```

1 #####
2 # CONSTANTS AND GLOBAL VARIABLES
3 #####
4 CONST MAX_WORKERS = 8
5 CONST MAX_PENDING_CONNECTIONS = 100
6 CONST HOST = "127.0.0.1"
7 CONST PORT = 3000
8
9 GLOBAL server_running = TRUE
10 GLOBAL PLAYER_ID_COUNTER = 0
11 GLOBAL GAME_ID_COUNTER = 0
12
13 #####
14 # CLASS DEFINITIONS
15 #####
16
17 CLASS Player
18     Attributes:
19         id = 0
20         connection = NULL
21         address = NULL
22         elixir = 0
23         is_active = TRUE
24         arena = 1
25         chests = []
26         request_queue = []
27
28     Methods:
29         FUNCTION Player(new_player_id, new_connection, new_address)
30             id = new_player_id
31             connection = new_connection
32             address = new_address
33             elixir = 0
34             is_active = TRUE
35             arena = 1
36             chests = []
37             request_queue = []
38         END Player
39
40         FUNCTION WAIT_FOR_REQUEST()
41             raw_data = connection.READ()
42             IF raw_data == EMPTY THEN
43                 RETURN NULL
44             END IF
45
46             parsed_request = PARSE_REQUEST(raw_data)
47             RETURN parsed_request
48         END WAIT_FOR_REQUEST
49
50         FUNCTION ADD_REQUEST(request)
51             request_queue.append(request)
52         END ADD_REQUEST
53
54         FUNCTION has_pending_requests()
55             IF request_queue.length > 0 THEN
56                 RETURN TRUE
57             ELSE

```

```

58         RETURN FALSE
59     END IF
60     END has_pending_requests
61
62     FUNCTION GET_REQUEST()
63         IF request_queue.length == 0 THEN
64             RETURN NULL
65         END IF
66
67         req = request_queue[0]
68         request_queue.remove_at(0)
69         RETURN req
70     END GET_REQUEST
71
72     FUNCTION SEND_MESSAGE(messageData)
73         connection.WRITE(messageData) # Stub
74     END SEND_MESSAGE
75 END Player
76
77
78 CLASS Entity
79     Attributes:
80         id = 0
81         type = ""
82         hp = 0
83         damage = 0
84         range = 0
85         position = {"x": 0, "y": 0}
86         side = ""
87
88     Methods:
89         FUNCTION Entity()
90             id = GENERATE_ENTITY_ID()
91             type = ""
92             hp = 0
93             damage = 0
94             range = 0
95             position = {"x": 0, "y": 0}
96             side = ""
97         END Entity
98
99         FUNCTION IS_DESTROYED()
100             IF hp <= 0 THEN
101                 RETURN TRUE
102             END IF
103             RETURN FALSE
104         END IS_DESTROYED
105
106         FUNCTION PROCESS_BEHAVIOR(game)
107             # Placeholder for AI or behavior
108         END PROCESS_BEHAVIOR
109
110         FUNCTION UPDATE_POSITION(game)
111             # Placeholder for movement logic
112         END UPDATE_POSITION
113 END Entity
114
115
116 CLASS Tower : Entity
117     Attributes:
118         owner = 0
119         opposing_player_id = 0
120

```

```
121  Methods:
122  FUNCTION Tower(owner_id, initial_hp, tower_side)
123      Entity() # Call parent constructor
124      owner = owner_id
125      hp = initial_hp
126      side = tower_side
127      opposing_player_id = 0
128  END Tower
129 END Tower
130
131
132 CLASS Game
133  Attributes:
134      id = 0
135      players = []
136      player_ids = []
137      entities = []
138      towers = []
139      event_dispatcher = NULL
140      is_terminated = FALSE
141      time_elapsed = 0
142      max_duration = 180
143
144  Methods:
145  FUNCTION Game()
146      id = 0
147      players = []
148      player_ids = []
149      entities = []
150      towers = []
151      event_dispatcher = NULL
152      is_terminated = FALSE
153      time_elapsed = 0
154      max_duration = 180
155  END Game
156
157  FUNCTION ADD_PLAYER(player)
158      players.append(player)
159      player_ids.append(player.id)
160  END ADD_PLAYER
161
162  FUNCTION INCREMENT_TIME(delta_time)
163      time_elapsed = time_elapsed + delta_time
164  END INCREMENT_TIME
165
166  FUNCTION TERMINATE()
167      is_terminated = TRUE
168  END TERMINATE
169
170  FUNCTION REPLENISH_ELIXIR(delta_time)
171      REPLENISH_ELIXIR(self, delta_time)
172  END REPLENISH_ELIXIR
173
174  FUNCTION BROADCAST_GAME_STATE()
175      state_snapshot = {
176          "type": "game_state",
177          "entities": [],
178          "time_elapsed": time_elapsed
179      }
180
181      FOR e IN entities DO
182          entitySnapshot = {
183              "id": e.id,
```

```

184         "hp": e.hp,
185         "position": e.position
186     }
187     state_snapshot["entities"].append(entitySnapshot)
188 END FOR
189
190 FOR p IN players DO
191     p.SEND_MESSAGE(state_snapshot)
192 END FOR
193 END BROADCAST_GAME_STATE
194
195 FUNCTION UPDATE(delta_time)
196     UPDATE_GAME(self, delta_time)
197 END UPDATE
198 END Game
199
200
201 CLASS Chest
202     Attributes:
203         type = ""
204         unlock_time = 0
205         rewards = []
206
207     Methods:
208         FUNCTION Chest()
209             type = ""
210             unlock_time = 0
211             rewards = []
212         END Chest
213
214         FUNCTION ASSIGN_UNLOCK_TIME()
215             CASE type OF
216                 "silver":
217                     unlock_time = 3 * 3600
218                 "gold":
219                     unlock_time = 8 * 3600
220                 "epic":
221                     unlock_time = 12 * 3600
222                 OTHERWISE:
223                     unlock_time = 1 * 3600
224             END CASE
225         END ASSIGN_UNLOCK_TIME
226
227         FUNCTION ALLOCATE_GOLD_REWARDS(player)
228             base_gold = 100
229             arena_factor = player.arena
230             total_gold = base_gold + (arena_factor * 10)
231             rewards.append({
232                 "type": "gold",
233                 "amount": total_gold
234             })
235         END ALLOCATE_GOLD_REWARDS
236
237         FUNCTION ALLOCATE_CARD_REWARDS(player)
238             ALLOCATE_CARD_REWARDS(self, player)
239         END ALLOCATE_CARD_REWARDS
240
241         FUNCTION ADD_CARDS(cards_list)
242             FOR c IN cards_list DO
243                 rewards.append({
244                     "type": "card",
245                     "card_id": c["id"],
246                     "rarity": c["rarity"]

```

```
247         })
248     END FOR
249 END ADD_CARDS
250 END Chest
251
252
253 CLASS Thread
254     Attributes:
255         functionToRun = NULL
256         args = []
257
258     Methods:
259         FUNCTION Thread(fn, fn_args)
260             functionToRun = fn
261             args = fn_args
262         END Thread
263
264         FUNCTION start()
265             # Stub for a new system thread
266         END start
267 END Thread
268
269
270 CLASS ThreadPool
271     Attributes:
272         size = 0
273         tasks = []
274
275     Methods:
276         FUNCTION ThreadPool(poolSize)
277             size = poolSize
278             tasks = []
279         END ThreadPool
280
281         FUNCTION init_workers()
282             # Stub
283         END init_workers
284
285         FUNCTION add_task(fn, fn_args)
286             tasks.append([fn, fn_args])
287         END add_task
288 END ThreadPool
289
290
291 CLASS PriorityQueue
292     Attributes:
293         internal_queue = []
294
295     Methods:
296         FUNCTION PriorityQueue()
297             internal_queue = []
298         END PriorityQueue
299
300         FUNCTION ADD(player, priority)
301             internal_queue.append({"player": player, "priority": priority})
302             # In practice, reorder by priority
303         END ADD
304
305         FUNCTION HAS_AVAILABLE_PAIRS()
306             IF internal_queue.length >= 2 THEN
307                 RETURN TRUE
308             END IF
309             RETURN FALSE
```

```

310     END HAS_AVAILABLE_PAIRS
311
312     FUNCTION GET_MATCHED_PLAYERS()
313         IF internal_queue.length < 2 THEN
314             RETURN (NULL, NULL)
315         END IF
316
317         internal_queue = SORT_BY_PRIORITY_DESC(internal_queue)
318         p1 = internal_queue[0]["player"]
319         p2 = internal_queue[1]["player"]
320         internal_queue.remove_at(0)
321         internal_queue.remove_at(0)
322
323         RETURN (p1, p2)
324     END GET_MATCHED_PLAYERS
325
326     FUNCTION REMOVE(player)
327         FOR i = 0 TO internal_queue.length - 1 DO
328             IF internal_queue[i]["player"].id == player.id THEN
329                 internal_queue.remove_at(i)
330                 BREAK
331             END IF
332         END FOR
333     END REMOVE
334 END PriorityQueue
335
336
337 CLASS EventDispatcher
338     Attributes:
339         events = []
340
341     Methods:
342         FUNCTION EventDispatcher()
343             events = []
344         END EventDispatcher
345
346         FUNCTION CREATE_EVENT(event_name, data)
347             events.append({"name": event_name, "data": data})
348         END CREATE_EVENT
349
350         FUNCTION EXECUTE_PENDING_EVENTS()
351             # Stub to process events
352             events = []
353         END EXECUTE_PENDING_EVENTS
354
355         FUNCTION SCHEDULE_EVENT(target, event_name)
356             events.append({"name": event_name, "target": target})
357         END SCHEDULE_EVENT
358 END EventDispatcher
359
360
361 #####
362 # GLOBAL OBJECTS
363 #####
364 GLOBAL GLOBAL_THREAD_POOL = new ThreadPool(MAX_WORKERS)
365 GLOBAL matchmaking_queue = new PriorityQueue()
366
367
368 #####
369 # 1. Main Server Loop
370 #####
371 FUNCTION RunServer()
372     server_socket = INITIALIZE_SERVER_SOCKET(HOST, PORT)

```

```

373 player_registry = {} # {player_id: Player}
374 active_games = {} # {game_id: Game}
375 event_dispatcher = new EventDispatcher()
376
377 GLOBAL_THREAD_POOL.init_workers()
378
379 START_THREAD(UpdateAllGames, [active_games, event_dispatcher])
380
381 WHILE server_running == TRUE DO
382     new_connections = server_socket.POLL_CONNECTIONS()
383     FOR connection_request IN new_connections DO
384         (connection, address) = server_socket.ACCEPT(connection_request)
385         player = REGISTER_PLAYER(connection, address)
386         player_registry[player.id] = player
387         ASSIGN_THREAD(HANDLE_PLAYER, [player])
388     END FOR
389
390     FOR each_player_id IN player_registry.keys DO
391         WHILE player_registry[each_player_id].has_pending_requests() == TRUE DO
392             pending_request = player_registry[each_player_id].GET_REQUEST()
393             HANDLE_PLAYER_REQUEST(
394                 player_registry[each_player_id],
395                 pending_request,
396                 matchmaking_queue,
397                 active_games,
398                 event_dispatcher
399             )
400         END WHILE
401     END FOR
402
403     PROCESS_MATCHMAKING(matchmaking_queue, active_games, event_dispatcher)
404     event_dispatcher.EXECUTE_PENDING_EVENTS()
405 END WHILE
406
407 CLEANUP_SERVER_RESOURCES(server_socket, player_registry, active_games)
408 END RunServer
409
410 #####
411 # 2. Player Handling and Requests
412 #####
413 FUNCTION HANDLE_PLAYER(player)
414     WHILE player.is_active == TRUE DO
415         request = player.WAIT_FOR_REQUEST()
416         IF request != NULL THEN
417             player.ADD_REQUEST(request)
418         END IF
419     END WHILE
420
421     CLEANUP_PLAYER(player)
422 END HANDLE_PLAYER
423
424 #####
425 # 3. Request Routing
426 #####
427 FUNCTION HANDLE_PLAYER_REQUEST(player, request, matchmaking_queue, active_games,
428     event_dispatcher)
429     CASE request.type OF
430         "matchmaking":
431             rank_value = request.data["rank"]
432             IF rank_value == NULL THEN
433                 rank_value = 0
434             END IF
435             matchmaking_queue.ADD(player, rank_value)

```

```

435     SEND_INFO(player, "You have been queued for matchmaking.")
436
437     "game_action":
438         game = GET_ACTIVE_GAME_FOR_PLAYER(player, active_games)
439         IF game != NULL THEN
440             action_type = request.data["action"]
441             IF action_type == "deploy" THEN
442                 card_id = request.data["card_id"]
443                 position = request.data["position"]
444                 IF card_id != NULL AND position != NULL THEN
445                     ProcessCardDeployment(game, player, card_id, position)
446                 ELSE
447                     SEND_ERROR(player, "Invalid deployment data.")
448                 END IF
449             ELSE
450                 game.UPDATE(0) # Stub
451                 game.EXECUTE_ACTION(player, request.data) # Placeholder
452             END IF
453         ELSE
454             SEND_ERROR(player, "You have no active game.")
455         END IF
456
457     "disconnect":
458         LOGOUT_PLAYER(player)
459         SEND_INFO(player, "You have disconnected.")
460
461     "custom_event":
462         event_data = request.data["event_data"]
463         IF event_data == NULL THEN
464             event_data = {}
465         END IF
466         event_dispatcher.CREATE_EVENT("CustomPlayerEvent", event_data)
467
468     OTHERWISE:
469         SEND_ERROR(player, "Unknown request type: " + request.type)
470     END CASE
471 END HANDLE_PLAYER_REQUEST
472
473 #####
474 # 4. Matchmaking Process
475 #####
476 FUNCTION PROCESS_MATCHMAKING(matchmaking_queue, active_games, event_dispatcher)
477     WHILE matchmaking_queue.HAS_AVAILABLE_PAIRS() == TRUE DO
478         (player1, player2) = matchmaking_queue.GET_MATCHED_PLAYERS()
479         IF player1 != NULL AND player2 != NULL THEN
480             new_game = INITIALIZE_NEW_GAME(player1, player2, event_dispatcher)
481             active_games[new_game.id] = new_game
482             NOTIFY_PLAYERS_OF_MATCH(player1, player2, new_game)
483         ELSE
484             BREAK
485         END IF
486     END WHILE
487 END PROCESS_MATCHMAKING
488
489 #####
490 # 5. Game Update Thread
491 #####
492 FUNCTION UpdateAllGames(active_games, event_dispatcher)
493     TICK_RATE = 30
494     FRAME_DURATION = 1.0 / TICK_RATE
495
496     WHILE server_running == TRUE DO
497         current_games = active_games.values()

```



```

498     FOR each_game IN current_games DO
499         IF each_game.is_terminated == FALSE THEN
500             each_game.UPDATE(FRAME_DURATION)
501             event_dispatcher.SCHEDULE_EVENT(each_game, "update_completed")
502         ELSE
503             active_games.pop(each_game.id, NULL)
504         END IF
505     END FOR
506
507     SLEEP(FRAME_DURATION)
508 END WHILE
509 END UpdateAllGames
510
511 #####
512 # 6. Game Update Logic
513 #####
514 FUNCTION UPDATE_GAME(game, delta_time)
515     game.INCREMENT_TIME(delta_time)
516
517     IF CHECK_GAME_END_CONDITIONS(game) == TRUE THEN
518         game.TERMINATE()
519         RETURN
520     END IF
521
522     game.REPLENISH_ELIXIR(delta_time)
523     UPDATE_ENTITIES(game)
524     game.BROADCAST_GAME_STATE()
525 END UPDATE_GAME
526
527 FUNCTION CHECK_GAME_END_CONDITIONS(game)
528     IF game.time_elapsed >= game.max_duration THEN
529         DETERMINE_GAME_WINNER_BY_HP(game)
530         RETURN TRUE
531     END IF
532
533     FOR tower IN game.towers DO
534         IF tower.hp <= 0 THEN
535             ASSIGN_WINNER(game, tower.opposing_player_id)
536             RETURN TRUE
537         END IF
538     END FOR
539
540     RETURN FALSE
541 END CHECK_GAME_END_CONDITIONS
542
543 FUNCTION ASSIGN_WINNER(game, winning_player_id)
544     game.winner_id = winning_player_id
545     game.is_terminated = TRUE
546     # Additional logic
547 END ASSIGN_WINNER
548
549 FUNCTION DETERMINE_GAME_WINNER_BY_HP(game)
550     best_score = -1
551     winning_player = NULL
552
553     FOR each_player IN game.players DO
554         total_hp = SUM_OF_ALL_TOWERS_HP(game, each_player) # Stub
555         IF total_hp > best_score THEN
556             best_score = total_hp
557             winning_player = each_player
558         END IF
559     END FOR
560

```

```

561     ASSIGN_WINNER(game, winning_player.id)
562 END DETERMINE_GAME_WINNER_BY_HP
563
564 FUNCTION REPLENISH_ELIXIR(game, delta_time)
565     FOR p IN game.players DO
566         current_elixir = p.elixir
567         regen_rate = game.get_elixir_regen_rate() # Stub
568         p.elixir = MIN(10, current_elixir + (regen_rate * delta_time))
569     END FOR
570 END REPLENISH_ELIXIR
571
572 FUNCTION UPDATE_ENTITIES(game)
573     entities_copy = game.entities[:]
574
575     FOR entity IN entities_copy DO
576         IF entity.IS_DESTROYED() == TRUE THEN
577             game.entities.remove(entity)
578         ELSE
579             entity.PROCESS_BEHAVIOR(game)
580             entity.UPDATE_POSITION(game)
581         END IF
582     END FOR
583 END UPDATE_ENTITIES
584
585 #####
586 # 7. Card Deployment Process
587 #####
588 FUNCTION ProcessCardDeployment(game, player, card_id, position)
589     IF VALIDATE_DEPLOYMENT_POSITION(game, player, position) == FALSE THEN
590         SEND_ERROR(player, "Invalid position for deployment.")
591         RETURN
592     END IF
593
594     card = player.GET_CARD_FROM_HAND(card_id) # Stub
595     IF card == NULL THEN
596         SEND_ERROR(player, "Card not found in hand.")
597         RETURN
598     END IF
599
600     IF VERIFY_ELIXIR_COST(player, card) == FALSE THEN
601         SEND_ERROR(player, "Insufficient elixir.")
602         RETURN
603     END IF
604
605     player.SPEND_ELIXIR(card.elixir_cost)
606     player.DISCARD_CARD(card_id)
607     player.DRAW_NEXT_CARD()
608
609     entity = CREATE_ENTITY_FROM_CARD(card, position, player.side)
610     game.entities.append(entity)
611
612     BROADCAST_DEPLOYMENT_UPDATE(game, player, card, position)
613 END ProcessCardDeployment
614
615 FUNCTION VALIDATE_DEPLOYMENT_POSITION(game, player, position)
616     IF position["x"] < 0 OR position["x"] > game.arena_width THEN
617         RETURN FALSE
618     END IF
619
620     IF player.side == "bottom" THEN
621         IF position["y"] < 0 OR position["y"] > (game.arena_height / 2) THEN
622             RETURN FALSE
623         END IF

```

```

624 ELSE
625     IF position["y"] < (game.arena_height / 2) OR position["y"] > game.arena_height THEN
626         RETURN FALSE
627     END IF
628 END IF
629
630 RETURN TRUE
631 END VALIDATE_DEPLOYMENT_POSITION
632
633 FUNCTION VERIFY_ELIXIR_COST(player, card)
634     IF player.elixir >= card.elixir_cost THEN
635         RETURN TRUE
636     ELSE
637         RETURN FALSE
638     END IF
639 END VERIFY_ELIXIR_COST
640
641 FUNCTION CREATE_ENTITY_FROM_CARD(card, position, side)
642     new_entity = new Entity()
643     new_entity.type = card.type
644     new_entity.hp = card.hp
645     new_entity.damage = card.damage
646     new_entity.range = card.range
647     new_entity.position = position
648     new_entity.side = side
649
650     RETURN new_entity
651 END CREATE_ENTITY_FROM_CARD
652
653 FUNCTION BROADCAST_DEPLOYMENT_UPDATE(game, player, card, position)
654     update_message = {
655         "type": "deployment",
656         "player_id": player.id,
657         "card_id": card.id,
658         "position": position
659     }
660
661     FOR p IN game.players DO
662         p.SEND_MESSAGE(update_message)
663     END FOR
664 END BROADCAST_DEPLOYMENT_UPDATE
665
666 #####
667 # 8. Target Selection Algorithm
668 #####
669 FUNCTION DetermineBestTarget(attacker, game_state)
670     potential_targets = game_state.GET_ENEMY_ENTITIES(attacker.side) # Stub
671     best_target = NULL
672     minimum_threat_score = INFINITY
673
674     FOR target IN potential_targets DO
675         threat_score = COMPUTE_THREAT_SCORE(attacker, target)
676         IF threat_score < minimum_threat_score THEN
677             minimum_threat_score = threat_score
678             best_target = target
679         END IF
680     END FOR
681
682     RETURN best_target
683 END DetermineBestTarget
684
685 FUNCTION COMPUTE_THREAT_SCORE(attacker, target)
686     DISTANCE_FACTOR = 1.0

```

```

687 HEALTH_FACTOR = 0.01
688 PRIORITY_FACTOR = 5.0
689
690 distance_val = CALCULATE_DISTANCE(attacker.position, target.position) # Stub
691 distance_weight = distance_val * DISTANCE_FACTOR
692 health_weight = target.hp * HEALTH_FACTOR
693 priority_weight = target.PRIORITY() * PRIORITY_FACTOR # Stub
694
695 total = distance_weight + health_weight + priority_weight
696 RETURN total
697 END COMPUTE_THREAT_SCORE
698
699 #####
700 # 9. Chest Reward System
701 #####
702 FUNCTION GenerateRewardChest(player, chest_type)
703     newChest = new Chest()
704     newChest.type = chest_type
705     newChest.ASSIGN_UNLOCK_TIME()
706     newChest.ALLOCATE_GOLD_REWARDS(player)
707     newChest.ALLOCATE_CARD_REWARDS(player)
708
709     player.chests.append(newChest)
710     SEND_REWARD_NOTIFICATION(player, newChest)
711 END GenerateRewardChest
712
713 FUNCTION ALLOCATE_CARD_REWARDS(chest, player)
714     card_distribution = DETERMINE_CARD_DISTRIBUTION(player.arena, chest.type)
715     available_cards = FETCH_AVAILABLE_CARDS(player.arena)
716
717     FOR (rarity, count) IN card_distribution.items DO
718         selected_cards = PICK_RANDOM_CARDS(available_cards, rarity, count)
719         chest.ADD_CARDS(selected_cards)
720     END FOR
721 END ALLOCATE_CARD_REWARDS
722
723 FUNCTION DETERMINE_CARD_DISTRIBUTION(arena, chest_type)
724     IF chest_type == "silver" THEN
725         RETURN {"common": 10, "rare": 2}
726     ELSE IF chest_type == "gold" THEN
727         RETURN {"common": 20, "rare": 5, "epic": 1}
728     ELSE
729         RETURN {"common": 5, "rare": 1}
730     END IF
731 END DETERMINE_CARD_DISTRIBUTION
732
733 FUNCTION FETCH_AVAILABLE_CARDS(arena)
734     RETURN [
735         {"id": "knight", "rarity": "common"},
736         {"id": "archer", "rarity": "common"},
737         {"id": "giant", "rarity": "rare"},
738         {"id": "dragon", "rarity": "epic"}
739     ]
740 END FETCH_AVAILABLE_CARDS
741
742 FUNCTION PICK_RANDOM_CARDS(cards, rarity, count)
743     same_rarity_cards = []
744     selected = []
745
746     FOR c IN cards DO
747         IF c["rarity"] == rarity THEN
748             same_rarity_cards.append(c)
749         END IF

```

```

750     END FOR
751
752     FOR i = 1 TO count DO
753         IF same_rarity_cards.length > 0 THEN
754             r = RANDOM_INDEX(0, same_rarity_cards.length - 1)
755             selected_card = same_rarity_cards[r]
756             selected.append(selected_card)
757         END IF
758     END FOR
759
760     RETURN selected
761 END PICK_RANDOM_CARDS
762
763 FUNCTION SEND_REWARD_NOTIFICATION(player, chest)
764     notification = {
765         "type": "chest_reward",
766         "chest_type": chest.type,
767         "estimated_unlock_time": chest.unlock_time
768     }
769
770     player.SEND_MESSAGE(notification)
771 END SEND_REWARD_NOTIFICATION
772
773 #####
774 # 10. Detailed Definitions of Referenced Functions
775 #####
776 FUNCTION INITIALIZE_SERVER_SOCKET(host, port)
777     sock = CREATE_SOCKET()
778     sock.BIND(host, port)
779     sock.LISTEN(MAX_PENDING_CONNECTIONS)
780     RETURN sock
781 END INITIALIZE_SERVER_SOCKET
782
783 FUNCTION server_socket.POLL_CONNECTIONS()
784     pending_connections = OS_SPECIFIC_POLL(self.internal_socket) # Stub
785     RETURN pending_connections
786 END server_socket.POLL_CONNECTIONS
787
788 FUNCTION server_socket.ACCEPT(connection_request)
789     (conn, addr) = OS_ACCEPT(connection_request) # Stub
790     RETURN (conn, addr)
791 END server_socket.ACCEPT
792
793 FUNCTION CREATE_SOCKET()
794     my_socket = {
795         "BIND": (host, port) -> None,
796         "LISTEN": (maxConn) -> None,
797         "CLOSE": () -> None,
798         "internal_socket": "OS_Socket_Object"
799     }
800     RETURN my_socket
801 END CREATE_SOCKET
802
803 FUNCTION ASSIGN_THREAD(functionToRun, args)
804     GLOBAL_THREAD_POOL.add_task(functionToRun, args)
805 END ASSIGN_THREAD
806
807 FUNCTION START_THREAD(functionToRun, args)
808     t = new Thread(functionToRun, args)
809     t.start()
810     RETURN t
811 END START_THREAD
812

```

```
813 FUNCTION REGISTER_PLAYER(connection, address)
814     PLAYER_ID_COUNTER = PLAYER_ID_COUNTER + 1
815     new_player_id = PLAYER_ID_COUNTER
816
817     new_player = new Player(new_player_id, connection, address)
818     SEND_INFO(new_player, "Welcome! Your player ID is " + STR(new_player_id))
819     RETURN new_player
820 END REGISTER_PLAYER
821
822 FUNCTION CLEANUP_PLAYER(player)
823     IF player.connection != NULL THEN
824         player.connection.CLOSE()
825     END IF
826     player.is_active = FALSE
827     matchmaking_queue.REMOVE(player)
828 END CLEANUP_PLAYER
829
830 FUNCTION GET_ACTIVE_GAME_FOR_PLAYER(player, active_games)
831     FOR game_id IN active_games.keys DO
832         game = active_games[game_id]
833         IF player.id IN game.player_ids THEN
834             RETURN game
835         END IF
836     END FOR
837     RETURN NULL
838 END GET_ACTIVE_GAME_FOR_PLAYER
839
840 FUNCTION SEND_ERROR(player, message)
841     error_packet = {
842         "type": "error",
843         "message": message
844     }
845     player.SEND_MESSAGE(error_packet)
846 END SEND_ERROR
847
848 FUNCTION SEND_INFO(player, message)
849     info_packet = {
850         "type": "info",
851         "message": message
852     }
853     player.SEND_MESSAGE(info_packet)
854 END SEND_INFO
855
856 FUNCTION LOGOUT_PLAYER(player)
857     player.is_active = FALSE
858     matchmaking_queue.REMOVE(player)
859 END LOGOUT_PLAYER
860
861 FUNCTION INITIALIZE_NEW_GAME(player1, player2, event_dispatcher)
862     GAME_ID_COUNTER = GAME_ID_COUNTER + 1
863
864     new_game = new Game()
865     new_game.id = GAME_ID_COUNTER
866     new_game.event_dispatcher = event_dispatcher
867     new_game.is_terminated = FALSE
868     new_game.time_elapsed = 0
869     new_game.max_duration = 180
870
871     new_game.ADD_PLAYER(player1)
872     new_game.ADD_PLAYER(player2)
873
874     new_game.towers = CREATE_DEFAULT_TOWERS(player1, player2)
875     FOR tower IN new_game.towers DO
```

```
876         new_game.entities.append(tower)
877     END FOR
878
879     RETURN new_game
880 END INITIALIZE_NEW_GAME
881
882 FUNCTION CREATE_DEFAULT_TOWERS(player1, player2)
883     tower1 = new Tower(player1.id, 3000, "bottom")
884     tower2 = new Tower(player1.id, 2000, "bottom")
885     tower3 = new Tower(player1.id, 2000, "bottom")
886     tower4 = new Tower(player2.id, 3000, "top")
887     tower5 = new Tower(player2.id, 2000, "top")
888     tower6 = new Tower(player2.id, 2000, "top")
889
890     tower1.opposing_player_id = player2.id
891     tower2.opposing_player_id = player2.id
892     tower3.opposing_player_id = player2.id
893     tower4.opposing_player_id = player1.id
894     tower5.opposing_player_id = player1.id
895     tower6.opposing_player_id = player1.id
896
897     tower_list = [tower1, tower2, tower3, tower4, tower5, tower6]
898     RETURN tower_list
899 END CREATE_DEFAULT_TOWERS
900
901 FUNCTION CLEANUP_SERVER_RESOURCES(server_socket, player_registry, active_games)
902     server_socket.CLOSE()
903
904     FOR pid IN player_registry.keys DO
905         CLEANUP_PLAYER(player_registry[pid])
906     END FOR
907
908     FOR gid IN active_games.keys DO
909         g = active_games[gid]
910         g.TERMINATE()
911     END FOR
912 END CLEANUP_SERVER_RESOURCES
913
914 #####
915 # 11. Error Handling & Validation
916 #####
917 FUNCTION RAISE_ERROR(message)
918     PRINT("ERROR: " + message)
919 END RAISE_ERROR
```

## 6.1 Algorithm Flowcharts

### 6.1.1 Main Server Loop Flowchart

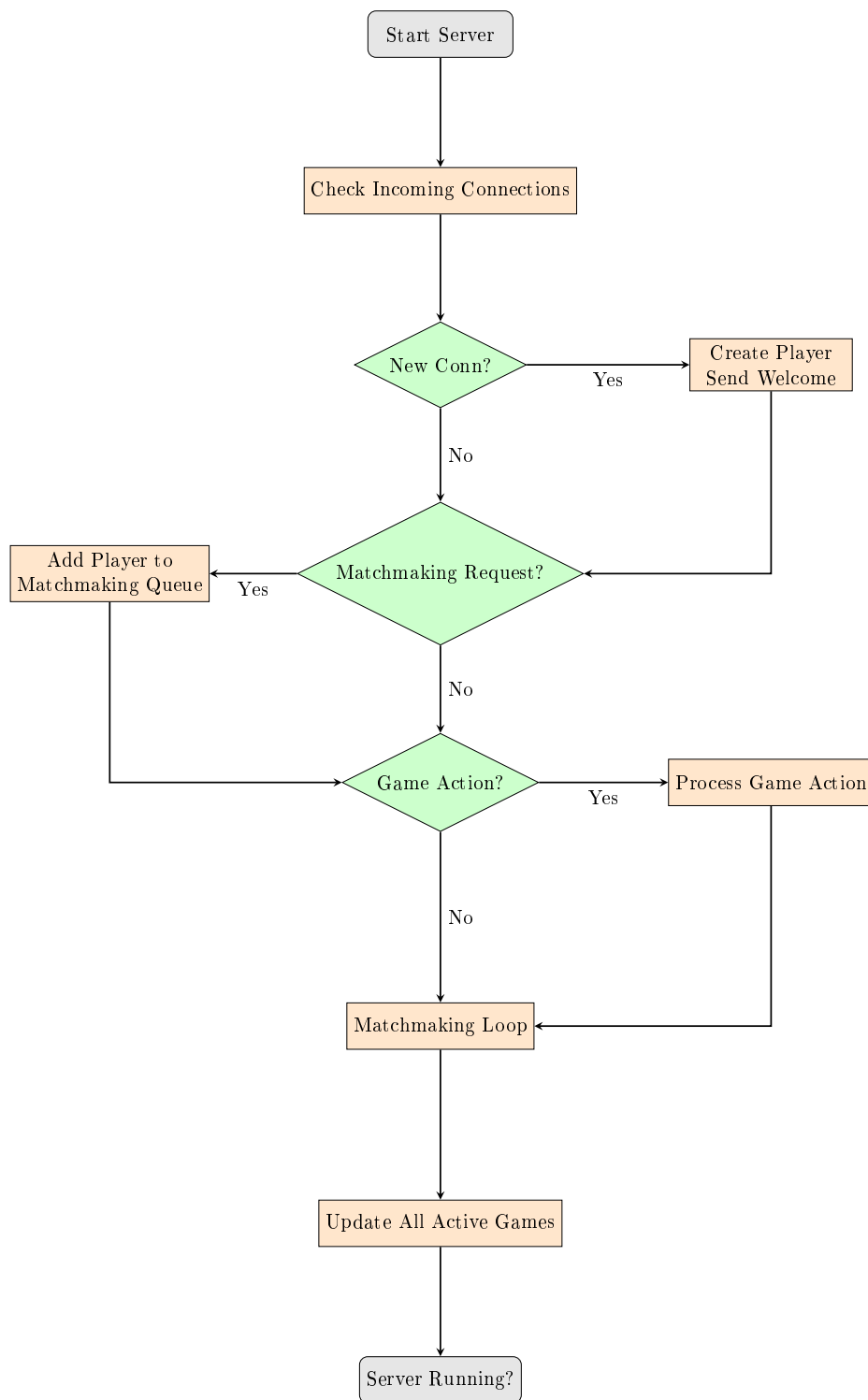


Figure 6: Main Server Loop Flowchart



### 6.1.2 Card Deployment Process Flowchart

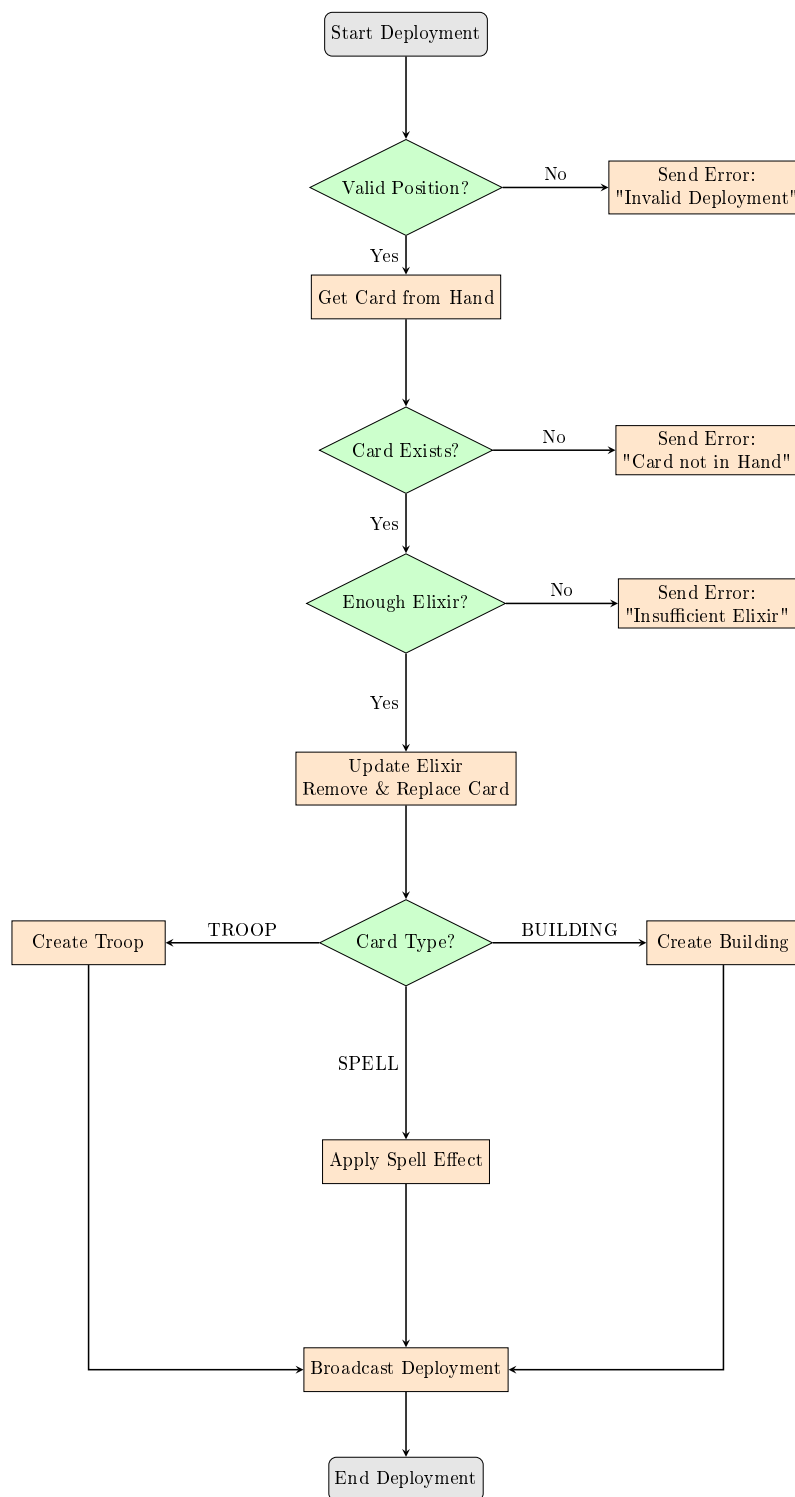


Figure 7: Card Deployment Flowchart

### 6.1.3 Battle Update Logic Flowchart

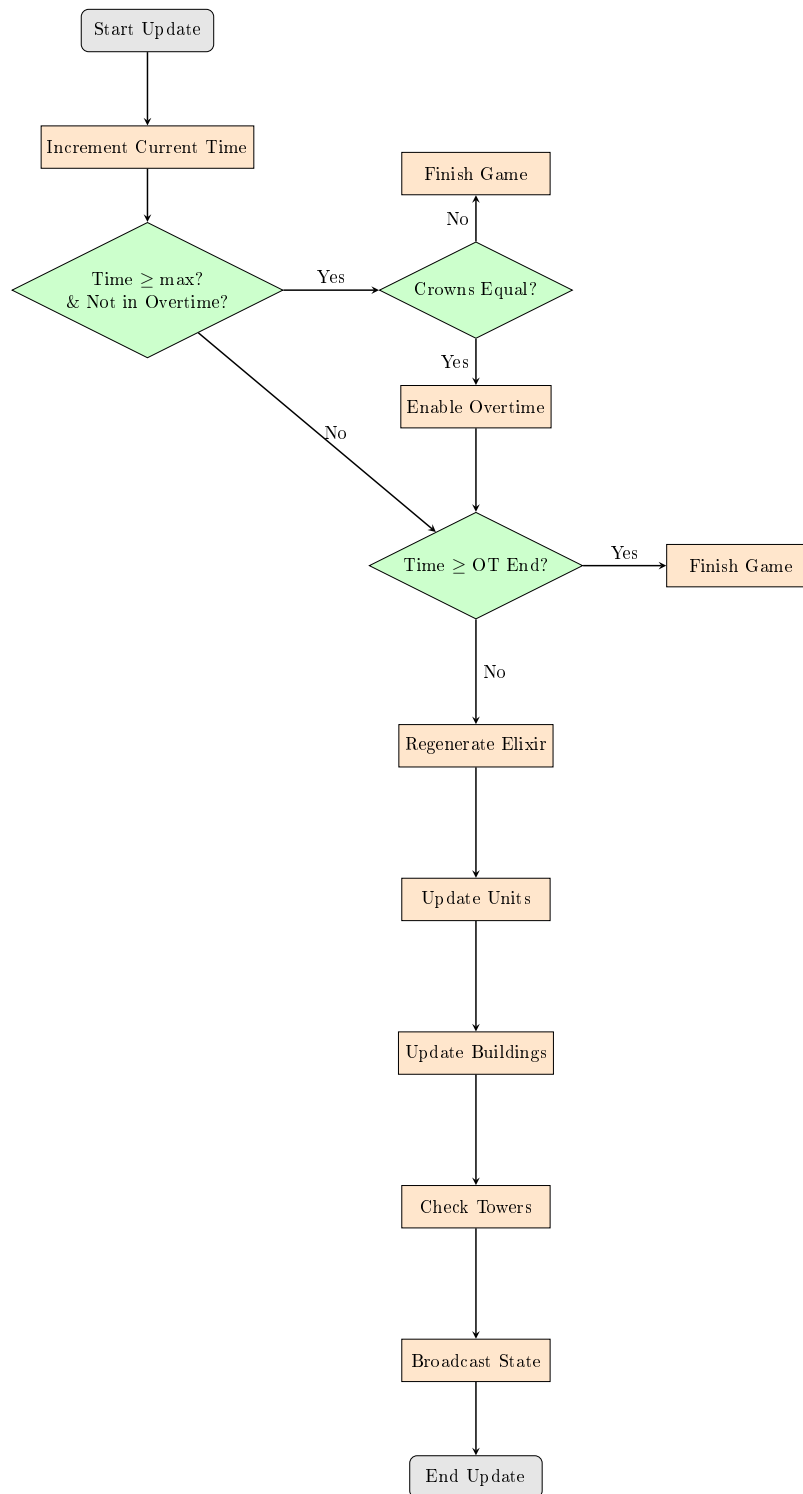


Figure 8: Battle Update Logic Flowchart

#### 6.1.4 Target Selection Algorithm Flowchart

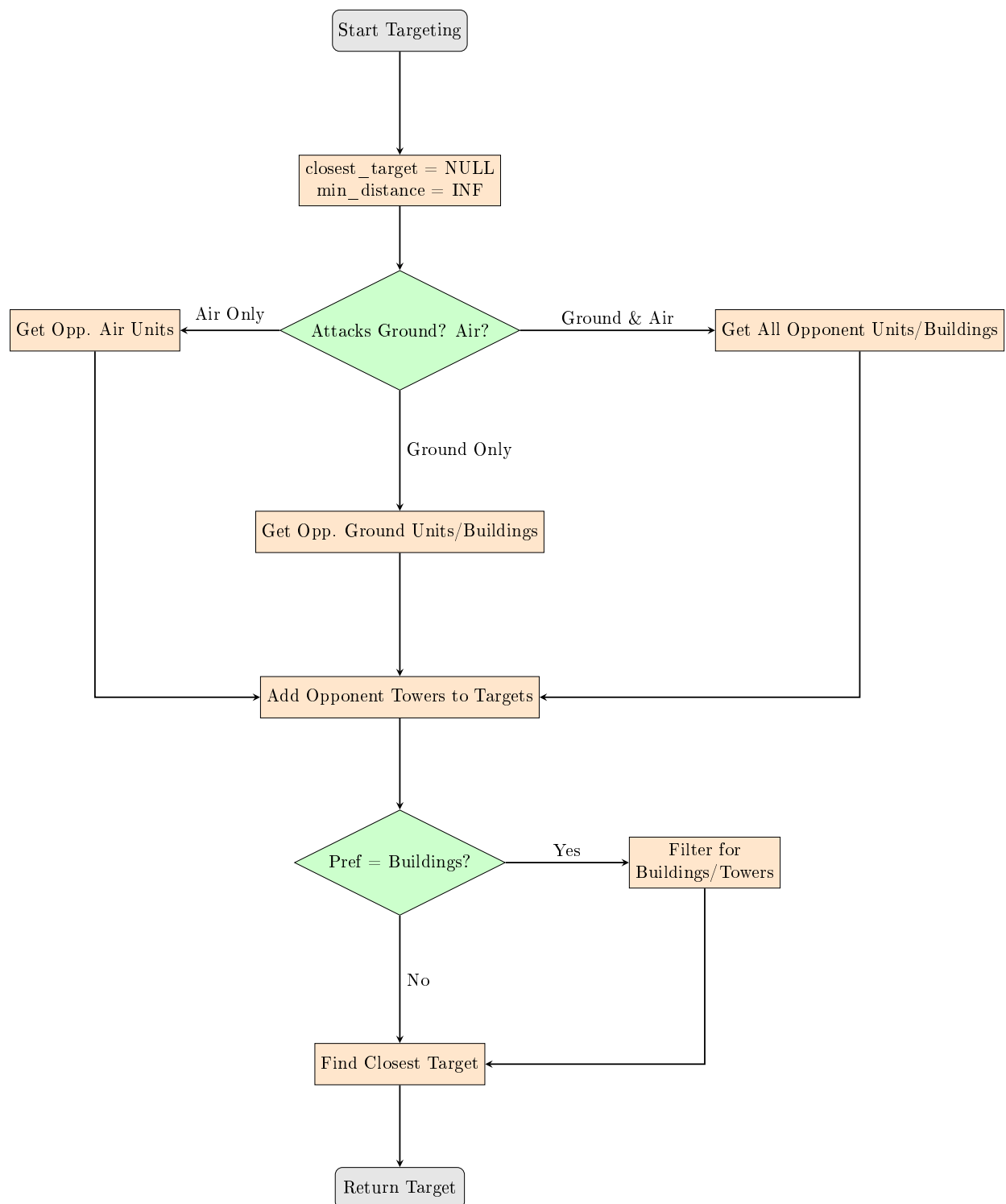


Figure 9: Target Selection Flowchart

### 6.1.5 Chest Reward System Flowchart

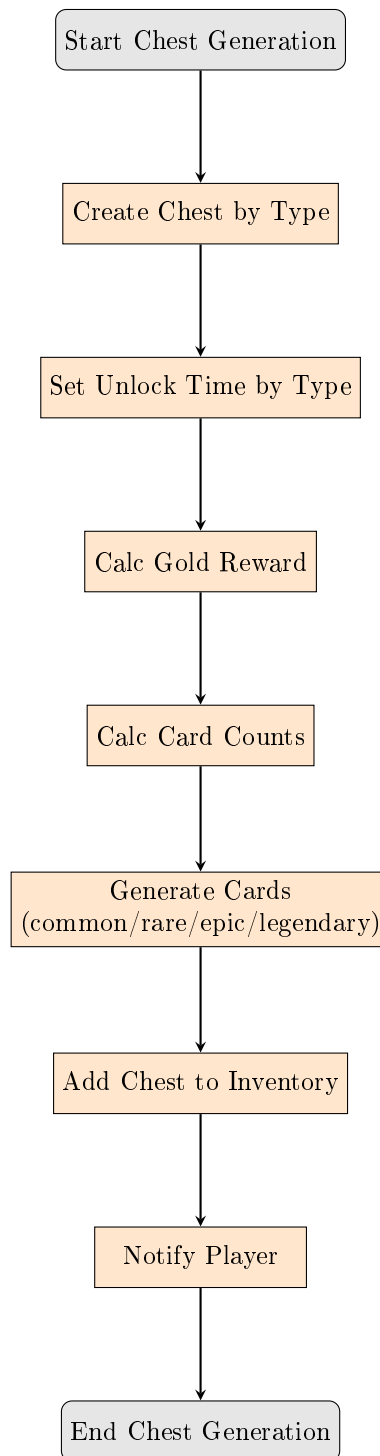


Figure 10: Chest Reward System Flowchart

## 7 Trace Tables

### 7.1 Main Server Loop

| Step | Condition                      | Action                         | player_registry          | active_games  |
|------|--------------------------------|--------------------------------|--------------------------|---------------|
| 1    | new_connections=[ConnA]        | REGISTER_PLAYER(ConnA)         | {1: PlayerA}             | {}            |
| 2    | PlayerA requests matchmaking   | matchmaking_queue.ADD(PlayerA) | {1: PlayerA}             | {}            |
| 3    | new_connections=[ConnB]        | REGISTER_PLAYER(ConnB)         | {1: PlayerA, 2: PlayerB} | {}            |
| 4    | PlayerB requests matchmaking   | matchmaking_queue.ADD(PlayerB) | same                     | {}            |
| 5    | PROCESS_MATCHMAKING finds pair | INITIALIZE_NEW_GAME            | same                     | {1001: Game1} |
| 6    | event_dispatcher.EXECUTE       | no pending events              | same                     | same          |

Table 1: Main Server Loop - Two Players Matching

### 7.2 Player Handling (HANDLE\_PLAYER)

| Step | Condition                              | Action                          | Outcome                |
|------|----------------------------------------|---------------------------------|------------------------|
| 1    | player.is_active = True                | WAIT_FOR_REQUEST()              | request = None         |
| 2    | request is None                        | loop repeats                    | handle_player in wait  |
| 3    | external event: player.is_active=False | exit loop                       | CLEANUP_PLAYER(player) |
| 4    | CLEANUP_PLAYER                         | close socket, remove from queue | player inactivated     |

Table 2: HANDLE\_PLAYER - Disconnect Scenario

### 7.3 Request Routing (HANDLE\_PLAYER\_REQUEST)

| Step | Input                             | Check / Action             | Outcome               |
|------|-----------------------------------|----------------------------|-----------------------|
| 1    | request.type="game_action"        | GET_ACTIVE_GAME_FOR_PLAYER | game=Game1            |
| 2    | action="deploy", card_id="knight" | ProcessCardDeployment      | successful deployment |
| 3    | else branch?                      | not triggered              | -                     |
| 4    | End                               | -                          | Request done          |

Table 3: Request Routing - Deploy Action

### 7.4 Matchmaking Process

| Step | Condition               | Action                | matchmaking_queue | active_games    |
|------|-------------------------|-----------------------|-------------------|-----------------|
| 1    | queue.has_pairs()=True  | get_matched_players() | empty after pop   | {}              |
| 2    | players valid           | initialize new game   | []                | {2001: GameObj} |
| 3    | queue.has_pairs()=False | break                 | []                | same            |

Table 4: Matchmaking - Single Pair Scenario

### 7.5 Game Update Thread (UpdateAllGames)

| Step | Games                                | Check                    | Action          | Result            |
|------|--------------------------------------|--------------------------|-----------------|-------------------|
| 1    | {G1(not terminated), G2(terminated)} | G1: not terminated       | G1.UPDATE()     | G1 remains active |
| 2    | G2: is_terminated=True               | remove from active_games | G2 removed      | {G1} left         |
| 3    | Sleep                                | 1 / TICK_RATE            | wait next frame | -                 |

Table 5: Game Update Thread - One Terminated, One Active

## 7.6 Game Update Logic (UPDATE\_GAME)

| Step | Condition                          | Action              | Outcome                                |
|------|------------------------------------|---------------------|----------------------------------------|
| 1    | time_elapsed=179, max_duration=180 | INCREMENT_TIME(1s)  | time_elapsed=180                       |
| 2    | check_end_conditions               | time_elapsed >= 180 | DETERMINE_GAME_WINNER_BY_HP, TERMINATE |
| 3    | after termination                  | skip further steps  | game ended                             |

Table 6: Game Update Logic - Timeout Scenario

## 7.7 Card Deployment Process

| Step | Condition               | Action                       | Result            |
|------|-------------------------|------------------------------|-------------------|
| 1    | position=(3,4)          | VALIDATE_DEPLOYMENT_POSITION | valid position    |
| 2    | card in hand            | yes                          | proceed           |
| 3    | player.elixir=6, cost=4 | verify cost                  | enough elixir     |
| 4    | SPEND_ELIXIR(4)         | discard card, draw next      | elixir=2          |
| 5    | CREATE_ENTITY           | new troop in game            | entity registered |
| 6    | BROADCAST_DEPLOYMENT    | all players see new troop    | done              |

Table 7: Card Deployment - Valid Scenario

## 7.8 Target Selection Algorithm

| Step | Targets          | Compute Score      | Min Score Target  |
|------|------------------|--------------------|-------------------|
| 1    | [T1, T2, T3]     | T1=10, T2=7, T3=12 | T2 is best so far |
| 2    | best_target = T2 | -                  | final = T2        |

Table 8: Target Selection - Closest Target Example